

Three Ways Input-to-State Mutation *Hurts*: A Byte-Level Account of the I2S-Anti Mechanisms

BlockerAnalyzer benchmark — I2S-anti family

2026-06-13

Abstract

In the BlockerAnalyzer benchmark, 41 decisive blocker branches are labeled *I2S-anti*: branches the input-to-state (I2S / cmplog) fuzzer *fails* to reach while a non-I2S arm (naive havoc or value-profile) succeeds. All three I2S-anti sub-mechanisms share one measured signature — on the real campaign corpus, the cmplog arm is *depleted* of the byte value the winning arm reaches — but they differ in *why*. This note defines the measurement at the byte level, then walks one specific branch per mechanism with its real gate offset(s), byte values, and corpus statistics.

1 The shared measurement: operand_enrichment

Every I2S-anti label is decided by one tool, `i2s_operand_availability.py` (registry name `operand_enrichment`), run directly on the two campaign corpora. The tool reads no source and assumes no mechanism; it measures bytes:

1. **Gate signature.** Compare the branch’s *resolving* (W) seeds against its *blocking* (L) seeds. The *gate offsets* are the byte positions at which W and L seeds systematically disagree (consensus purity $\geq$ threshold). At each gate offset, the W-consensus byte is the **target** pattern, the L-consensus byte is the **decoy** pattern.
2. **Corpus fractions.** For each fuzzer $F \in \{\text{cmplog}, \text{naive}\}$, scan F ’s saved corpus and compute `target_fracF` (fraction whose head matches the full target pattern at all gate offsets) and `decoy_fracF` (fraction matching the decoy pattern).
3. **Signed enrichment.**

$$\text{signed_target_enrich} = \log_2 \frac{\text{target_frac}_{\text{cmplog}}}{\text{target_frac}_{\text{naive}}}, \quad \text{decoy_enrich} = \log_2 \frac{\text{decoy_frac}_{\text{cmplog}}}{\text{decoy_frac}_{\text{naive}}}.$$

A value `signed_target_enrich` < 0 means cmplog’s corpus contains the target byte *less* often than naive’s — I2S is steering bytes *away* from the value that resolves the branch. All three labels require `signed_target_enrich` < -0.3 ; they are separated by the decisive *shape* (which fuzzers win/lose) and by *what* the decoy is.

2 Mechanism 1 — target_depletion: a rival operand wins the harvest

Example branch: `libpng_3977` (validated, $n = 23$ for this label).

Table 1: The three I2S-anti mechanisms at a glance. Shape code is over [cmplog, vp, vpc, naive] with W=win / L=loss / _=non-decisive.

label	shape	decoy is...	rule
i2s_decoy_substitution_target_depletion	LW_W	a competing real operand	signed_target_enrich
i2s_anti_structural_byte_corruption	LWWW	— (blind overwrite)	signed_target_enrich
i2s_decoy_substitution_overfit	LWLW	a structural “stay-valid” literal	signed_target_enrich

The gate is a single byte. `operand_enrichment` found `gate_offsets = [236]`, `gate_width = 1`. The branch is the `CHRM` (chromaticity) chunk handler comparing a parsed fixed-point value to the error sentinel:

Listing 1: `/src/libpng/pngutil.c:1283`, `OSS_FUZZ_png_handle_CHRM`. The gate byte at input offset 236 feeds `xy.blueX`.

```

1  if (xy.redx == PNG_FIXED_ERROR ||
2      xy.redy == PNG_FIXED_ERROR ||
3      xy.greenx == PNG_FIXED_ERROR ||
4      xy.greeny == PNG_FIXED_ERROR ||
5      xy.blueX == PNG_FIXED_ERROR || /* <-- decisive branch 3977 */
6      xy.bluey == PNG_FIXED_ERROR)
7      /* error path: chunk rejected */

```

At offset 236 the resolving seeds carry the **target byte** `0x00`; the blocking seeds carry the **decoy byte** `0x65` (ASCII ‘e’). The decoy is not random: it is the exponent character of ASCII numeric literals (“1.0e0”, “65.535”) that `cmplog` harvests from *other* string/number comparisons in the PNG parser and splices in via `I2SRandReplace`:

Listing 2: Hex window over bytes 228–244 around the gate byte (in brackets at offset 236). Top: a resolving (W) seed. Bottom: a blocking (L) `cmplog` seed — note the embedded ASCII float 1.0e0 whose e (`0x65`) lands exactly on the gate.

```

W (resolving, naive) : 02 02 02 02 02 75 30 00 [00] ea 60 20 00 79 98 00
00
L (blocking, cmplog) : 2f 62 61 72 00 31 2e 30 [65] 30 00 36 35 2e 35 33
35
                        ( "1.0e0" )                ( "65.535" )

```

Table 2: `libpng_3977` operand.enrichment metrics (`--sample 8000 --head 256`). `cmplog` is depleted of the target `0x00` and enriched in the decoy `0x65`.

quantity	cmplog	naive
corpus size scanned	4561	4447
target_frac (byte 0x00 @236)	0.0886	0.1565
decoy_frac (byte 0x65 @236)	0.6290	0.3929
signed_target_enrich = $\log_2(0.0886/0.1565) = -0.821$ (target depleted)		
decoy_enrich = $\log_2(0.6290/0.3929) = +0.679$ (decoy enriched)		

Reading: a real, abundant competing operand (the textual `0x65`) exists at the same byte position. `cmplog` logs it and `RandReplaces` it in, *pinning* 63% of its corpus to the decoy and dropping the

target 0x00 to 8.9% (vs. 15.7% for naive, which never harvests the decoy and drifts onto the target freely). The extra rule clause `skip != 'no_gate_signature'` guarantees a genuine two-operand competition, not noise.

3 Mechanism 2 — byte_corruption: blind RandReplace clobbers an assembled byte

Example branch: `harfbuzz_5831` (validated; server-a target).

Shape LWWW: `cmplog` is the *sole* loser (naive, `value_profile`, and `vpc` all win) — the clean fingerprint of “only the I2S mutator breaks this.” Here there is no clever rival operand. Naive *assembles* the correct structural byte; `cmplog`’s `I2SRandReplace` blindly overwrites that exact position with a logged value, *destroying* the byte rather than redirecting it to a single dominant decoy.

Table 3: `harfbuzz_5831` operand_enrichment metrics. The target byte is depleted in `cmplog` (`signed_target_enrich` < 0); decoy fractions are small in absolute terms (diffuse corruption), not one dominant rival.

quantity	<code>cmplog</code>	naive
<code>target_frac</code>	0.2113	0.3799
<code>decoy_frac</code>	0.0293	0.0030
<code>signed_target_enrich</code> = -0.882 (target byte clobbered out of <code>cmplog</code> ’s corpus)		
<code>decoy_enrich</code> = +2.64 (weak, low-absolute — diffuse, not a single decoy)		

Reading: the structural byte is present in 38% of naive’s corpus but only 21% of `cmplog`’s — I2S keeps overwriting it. Contrast with Mechanism 1: there the decoy is a *single abundant* value (`decoy_frac`= 0.63); here the decoy mass is small and scattered (`decoy_frac`= 0.029), the signature of *corruption* (random clobber) rather than *competition* (capture by one rival).

4 Mechanism 3 — overfit: capture by a “stay-valid” literal, never exploring the free byte

Example branch: `harfbuzz_5323` (validated; server-a target).

Shape LWLW: both I2S carriers (`cmplog` and, by the shape, the `value_profile` arm) lose; the winning gate is a *free byte* that naive/VP can hit by random mutation. `cmplog` instead gets *captured* by a *structural literal* elsewhere in the input — it keeps splicing that literal to keep the input well-formed, and as a result *never randomizes* the free-byte target. It “overfits” to structural validity.

Reading: this is the strongest anti-signal in the family. `cmplog` spends 51% of its corpus re-asserting the structural literal (`decoy_frac`_{`cmplog`} = 0.5075 vs. 0.0004 for naive) and collapses the winning free byte to 8% (vs. 51% for naive). Where Mechanism 1 is *two operands competing* and Mechanism 2 is *one byte being corrupted*, Mechanism 3 is a *validity trap*: I2S is “succeeding” at keeping the input structurally valid and thereby starving the one position that actually matters.

Table 4: `harfbuzz_5323` operand_enrichment metrics — the extreme case. `cmplog`’s corpus is half the structural decoy; `naive`’s has almost none.

quantity	<code>cmplog</code>	<code>naive</code>
<code>target_frac</code>	0.0816	0.5083
<code>decoy_frac</code>	0.5075	0.0004
<code>signed_target_enrich</code> = −2.64 (target free-byte present in 51% of <code>naive</code> vs 8% of <code>cmplog</code>)		
<code>decoy_enrich</code> = +10.4 (structural literal dominates <code>cmplog</code> , near-absent in <code>naive</code>)		

Table 5: The discriminating contrast. All three have `signed_target_enrich` < 0 (target depleted in `cmplog`); the *decoy* statistics separate them.

	M1 depletion <code>libpng_3977</code>	M2 corruption <code>harfbuzz_5831</code>	M3 overfit <code>harfbuzz_5323</code>
shape	LW_W	LWWW	LWLW
<code>signed_target_enrich</code>	−0.82	−0.88	−2.64
<code>decoy_enrich</code>	+0.68	+2.64	+10.4
<code>decoy_frac</code> <code>cmplog</code>	0.629	0.029	0.508
<code>decoy_frac</code> <code>naive</code>	0.393	0.003	0.0004
decoy character	one abundant rival	diffuse / none	one structural literal
winner gate	competing operand	assembled byte	free byte
one-line	rival wins harvest	byte clobbered	validity trap

5 Side-by-side

Provenance note. `libpng_3977` is a server-sB target; its gate offset, target/decoy bytes, and seed windows above were extracted directly from the on-disk campaign corpus. `harfbuzz_5831` and `harfbuzz_5323` are server-a targets; their metric values are taken verbatim from `step5b_new_v3/*/assignments_s4` but their per-offset byte windows are computed on server a’s corpus and are not reproduced here.